# Sparse and Long-Context Attention

pig7selene

September 5, 2026

**Abstract**

Long contexts make dense attention expensive because they enlarge both the set of pairwise interactions and the persistent state carried through autoregressive inference. This chapter develops sparse attention as an architectural restriction on which token pairs may interact, distinguishing it from FlashAttention's efficient execution of the same dense connectivity. It examines local, sliding-window, global, block-sparse, strided, dilated, and random patterns; uses Sparse Transformer, Longformer, and BigBird as representative designs; and explains how sparse connectivity, positional extension, KV Cache state, and retrieval jointly determine a model's usable long-context capability.

## 1 Introduction

Dense causal Self-Attention gives each token direct access to every earlier position. This flexible connectivity is one reason decoder-only Transformers can condition generation on a long prefix, but it scales poorly with sequence length. For a sequence of length $T$, the legal score structure contains on the order of $T^2$ pairwise interactions. Long contexts consequently pressure attention compute, intermediate memory, KV Cache capacity, memory bandwidth, and inference latency at once.

Chapter 3 develops the dense attention rule and causal mask; Chapter 24 derives the persistent KV Cache cost; and Chapter 30 separates FlashAttention-style execution from KV-representation choices such as MQA, GQA, and MLA. This chapter changes a different object: the **connectivity pattern**. Sparse attention computes only selected query-key pairs. It can substantially reduce the number of interactions, but it may also deny a token direct access to useful distant evidence. The result is an architectural quality-efficiency trade-off, not a free kernel substitution.

Let $T$ be sequence length and let $i$ and $j$ index query and key positions. We use $w$ for a local-window width, $g$ for the number of global positions, and $r$ for a block's token width where needed. The chapter assumes causal decoder-only attention unless a representative encoder-oriented design is explicitly named. Sparse-attention patterns are reusable ideas, but their exact masks, kernels, and training objectives must be stated before their asymptotic claims can be interpreted.

## 2 Dense and Sparse Connectivity

An attention mask can be described by a binary connectivity indicator

$$M_{i,j} \in \{0,1\}, \quad M_{i,j} = 1 \ \text{ if token } \ i \ \text{ may attend to token } \ j. \tag{1}$$

For dense causal attention, $M_{i,j} = 1$ whenever $j \le i$. For sparse attention, only a structured subset of those causal pairs is allowed. The masked score operation still uses the scaling and rowwise Softmax of Chapter 3, but the normalization domain is now the allowed neighborhood:

$$A_{i,j} = \frac{M_{i,j} \exp\left(S_{i,j}\right)}{\sum_{u=1}^{T} M_{i,u} \exp\left(S_{i,u}\right)}. \tag{2}$$

When $M_{i,j} = 0$, the position contributes zero probability. The formula assumes every query has at least one permitted key; causal masks normally retain the diagonal. An implementation may express the same rule through a large negative additive mask or a pattern-aware kernel rather than by explicitly multiplying by $M$.

This changes the attention graph itself. **FlashAttention** retains dense legal connectivity and improves its execution by tiling and online Softmax. **Sparse attention** omits selected interactions from the model's computation. The two ideas can be combined: a supported kernel can execute a structured sparse pattern efficiently, but an efficient dense kernel does not make a sparse architecture, and a sparse mask does not guarantee a fast implementation.

# 3 Local and Sliding-Window Attention

The simplest sparse pattern gives every token a fixed local history. With causal window width $w$, a token at position $i$ may attend approximately to

$$\max(1, i - w + 1) \le j \le i. \tag{3}$$

The number of attended keys per query is bounded by $w$, apart from sequence boundaries. The total number of score interactions is then on the order of $Tw$ rather than $T^2$, under the assumption that $w$ does not grow with $T$. Local attention is appropriate when most useful dependencies are nearby, but it creates a direct-access boundary: a token cannot inspect an arbitrary far-away position in one layer.

**Sliding-window attention** applies such overlapping neighborhoods continuously along a sequence. Neighboring queries share most of their allowed keys, producing a banded attention pattern rather than independent fixed chunks. This avoids artificial disconnections at segment boundaries, but does not restore dense global access. In an autoregressive model, every window must still respect $j \le i$; a symmetric window suitable for a bidirectional encoder is not automatically a valid causal generation mask.

## 3.1 Receptive Field and Information Propagation

The **receptive field** of a representation is the set of input positions that can influence it through the stack. Under a simple causal local window, information can propagate from a position to later positions one local hop per layer. Ignoring nonlinear transformations and boundary effects, after $\ell$ layers an output can be influenced by positions roughly $\ell w$ steps earlier through repeated relay paths. This is an effective, multi-layer reach, not direct one-layer access.

The distinction matters for long-range reasoning. A deep local stack can carry information across a document, but the signal must pass through intermediate representations and may compete with other information along the route. It is not equivalent to allowing every query to select any prior token directly. Window size, depth, training distribution, and the placement of global connections jointly determine whether this indirect path is useful for a particular dependency.

# 4 Global and Structured Sparse Patterns

Local attention can be augmented with a small set of global positions. A global token may attend broadly, be attended to broadly, or both, according to the declared mask. Special classification positions, task-defined locations, or selected content tokens are possible choices. If most tokens use a width-$w$ local neighborhood and $g$ global positions participate broadly, the interaction count is commonly on the order of

$$Tw + Tg, \tag{4}$$

when $w$ and $g$ are fixed relative to $T$. The global positions create short paths between distant regions without giving every ordinary token a dense row. Their semantics must be explicit: a global token that reads the whole document but is never visible to other tokens solves a different communication problem from one that is globally bidirectional.

## 4.1 Block, Strided, and Dilated Patterns

**Block-sparse attention** partitions the conceptual $T \times T$ matrix into $r \times r$ blocks and computes only an allowed subset of blocks. This is often more compatible with accelerator kernels than arbitrary element-wise sparsity because a permitted block contains regular dense matrix work. The realized

speedup depends on block size, layout, padding, occupancy, and kernel support; fewer nonzero entries alone do not imply proportional wall-clock improvement.

Other patterns choose long-range edges in a structured way. **Strided attention** permits regularly spaced positions, supplying periodic long-distance paths. **Dilated attention** increases the spacing between reachable positions across heads or layers, expanding reach without making every row dense. **Fixed global positions** devote a known small set of tokens to communication. **Random sparse connections** add irregular graph edges that can shorten paths between distant regions. These choices differ in inductive bias: a regular pattern is easier to reason about and often easier to execute, whereas random or learned-looking connections can improve graph connectivity but complicate reproducibility and kernel design.

# 5 Representative Sparse Architectures

Early Sparse Transformer work factorized the attention matrix into structured patterns to reduce the number of pairwise interactions while retaining paths for long-range information propagation [1]. The important general lesson is not one particular image- or byte-model layout. A sparse pattern must be evaluated as a graph: its edge count controls nominal work, while its path structure controls which information can travel where and how many layers that travel requires.

Longformer is a representative local-global design. Its sliding-window attention gives ordinary tokens local neighborhoods, while task-motivated global attention gives selected positions broad connectivity [2]. This arrangement was designed for long-document processing, where a small number of task-relevant positions can mediate document-wide exchange. Whether a global position should be a special token, a query token, or content selected at runtime depends on the task and changes the model interface.

BigBird combines local, random, and global attention [3]. Local edges preserve nearby structure, global edges supply broad communication, and random edges improve graph connectivity without making the full matrix dense. The original work establishes theoretical properties under its stated pattern and assumptions; those results should not be read as a guarantee that any arbitrary sparse mask preserves dense-attention quality for a particular causal LLM or deployment workload.

| Pattern | Allowed connections | Interaction scale | Main limitation |
|---|---|---|---|
| Dense causal | Every previous token. | $O(T^2)$ | Largest compute and attention-state pressure. |
| Local / sliding window | A width-$w$ causal neighborhood. | $O(Tw)$ for fixed $w$ | No direct access to distant tokens. |
| Local–global | Local neighborhoods plus $g$ broad positions. | $O(Tw + Tg)$ for fixed $w, g$ | Global-token choice becomes part of the task design. |
| Block-sparse | A declared subset of $r \times r$ blocks. | Pattern-dependent | Kernel efficiency can lag nominal sparsity. |
| Mixed structured | Local plus strided, dilated, random, or global edges. | Pattern-dependent | Connectivity and implementation are harder to validate. |

Table 1: Sparse-attention complexity depends on fixed pattern parameters. The table counts permitted attention interactions, not all runtime costs such as projections, KV state, or kernel overhead.

# 6 Long Context in Decoder-Only Models

Sparse-attention literature includes many bidirectional encoders, but decoder-only generation imposes additional constraints. Every connection must remain causal, Prefill must construct the appropriate cache state, Decode must extend it token by token, and the runtime must support the

pattern at the model's actual head dimensions and batch regime. A mask that is useful for long-document classification may need material changes before it can serve an autoregressive language model.

Modern long-context LLMs consequently take several paths. Some retain dense attention while relying on efficient kernels, better positional treatment, GQA or MLA, and KV Cache engineering. Some use sliding-window or other partially structured attention in selected layers. Others combine several mechanisms. Sparse attention is therefore one architectural lever for long context, not a definition of long-context capability. Chapter 30 explains how FlashAttention, MQA, GQA, and MLA address different execution or state bottlenecks that can coexist with a sparse connectivity choice.

### 6.1 KV Cache Does Not Disappear

Reducing attention edges does not automatically remove persistent decoding state. A straightforward decoder implementation may still store per-token Keys and Values for every layer, even if a later query reads only a subset. The ideal dense-cache payload remains proportional to

$$2BLTH_{\mathrm{kv}}d_h b, \tag{5}$$

subject to the actual architecture's cache representation. Sparse connectivity can lower read traffic for a Decode step, but cache capacity may still grow linearly with $T$. GQA reduces $H_{\mathrm{kv}}$; MLA changes the stored representation; KV quantization reduces effective $b$; eviction and compression change what historical state remains. Chapter 24 develops those mechanisms. Long-context systems must identify whether their active limitation is score computation, intermediate IO, persistent capacity, cache bandwidth, or a different component rather than assuming that sparse attention resolves all of them.

## 7 Position and Context Extension

Increasing computational context capacity does not guarantee that a checkpoint can use farther positions. Chapter 3 shows how RoPE makes attention scores depend on relative displacement. A model trained chiefly on shorter sequences can nevertheless behave poorly beyond that distribution, even when its mask and memory allocation accept the larger input. It is useful to distinguish three properties: **computational context capacity** is the maximum sequence that the architecture and runtime can process; **positional extrapolation** is behavior at position indices outside the training regime; and **learned long-context capability** is effective use of relevant evidence at those positions.

RoPE-based extension methods modify or rescale how position indices enter the rotary transformation. Position interpolation, for example, maps a longer target range into a position range closer to the one seen by the original model and is paired with long-context adaptation in the referenced study [4]. Frequency rescaling and related strategies pursue the same general aim: avoid treating a new positional range as if the original training distribution made it automatically familiar. The specific mechanism, fine-tuning data, and evaluation length are part of the claim; a larger configured maximum alone is not evidence of robust long-context use.

Practical context extension can combine additional long-sequence training, positional scaling, efficient attention kernels, reduced KV-head count, cache management, and structured sparsity. These techniques address different constraints and can interfere. For example, a model may accept a large positional index but run out of cache capacity under concurrency; a sparse pattern may save attention work while long-context training remains insufficient; and a successful kernel benchmark at one length may not predict evidence use at another. No single mechanism defines a usable context window.

## 8 Retrieval and Long Context

Long context and Retrieval-Augmented Generation (RAG) overlap but do not solve the same problem. Long context places more material directly in the model input. RAG selects relevant external units before generation. A large context window can accommodate more source material, but an entire corpus may still be too large, and irrelevant content still consumes Prefill work, context budget, and

model attention. Chapter 32 formalizes the retrieval and context-construction boundaries; Chapters 33–39 develop the selection and evaluation machinery.

Conversely, retrieval does not eliminate the value of long context. A selected set of documents may require retaining a long chain of evidence, a large codebase region, or multiple conflicting source passages. The useful system design is often a composition: retrieval controls corpus-scale selection, while the context window and attention architecture determine how much selected evidence can be processed together. Both must be evaluated against the actual evidence distribution and answer task.

## 9 Quality–Efficiency Trade-offs

Dense attention provides flexible direct connectivity but incurs the largest pairwise workload. Sparse patterns reduce permitted interactions by construction, so a relevant distant position can be omitted from a query's neighborhood or be reachable only after several layers. The correct design depends on the task's dependency lengths, the context-length distribution, model depth, training procedure, expected generation regime, and available kernels and hardware. A window that works for predominantly local syntax can be inadequate for a document-level reference, while a broad global pattern can dilute the intended computational saving.

The comparison must also avoid a common systems error: asymptotic interaction count is not latency. Projection layers, cache reads, model weights, block padding, irregular access, kernel launches, batching, and scheduling can dominate a measured path. Sparse attention may be beneficial at lengths where dense attention is untenable yet add overhead at short lengths. It should be compared with a dense baseline under the same model quality target, context length, precision, batch regime, and serving objective.

## 10 Failure Modes

A local window can be too small for a required dependency, and repeated layers may not repair the resulting information bottleneck. Global tokens can be poorly selected, overloaded, or semantically incompatible with a task. Random or structured edges can supply ineffective connectivity for the data distribution, while an apparently sparse pattern can fall back to a slow dense or unsupported kernel. Block padding and shape constraints can erase the expected advantage. These are architecture and systems failures, not merely hyperparameter choices.

Long-context failures cross the same boundaries. Position extrapolation can fail even when the sparse mask admits a long sequence. Long-sequence adaptation data can be unrepresentative. KV Cache capacity can become the active limitation after score computation is reduced. Excessive context can distract the generator, amplify irrelevant material, or give a nominal context limit far beyond the effective usable one. A regression suite should test evidence at varying distances, not only whether the runtime accepts a long input without allocation failure.

## 11 Implementation Contracts

The connectivity contract must define causal direction, every allowed edge family, local-window convention, global-token semantics, block size and indexing, boundary behavior, padding treatment, and the mapping from logical positions to positional indices. Tests should compare a sparse kernel with a dense reference restricted to the same mask, including first and final windows, global-token cases, partially filled blocks, packed sequences, and sequences near the maximum declared length. A dense fallback must preserve the same mask; falling back to unrestricted attention silently changes model behavior.

The long-context contract must record positional-scaling method, position range, tokenizer and serialization rules, KV Cache representation, cache dtype, cache policy, supported kernel paths, and the conditions that trigger fallback. It should separately measure attention compute, temporary

workspace, persistent cache, Prefill latency, Decode latency, and quality at several evidence distances. Regression tests must include long-range retrieval of an inserted fact, multi-step dependencies, adversarial irrelevant context, and the expected concurrency regime. These conditions distinguish a claimed configured context length from an auditable capability.

## 12 Summary

Sparse attention changes which token pairs can interact. Local, sliding-window, global, block-sparse, strided, dilated, and random patterns can reduce the number of permitted score interactions below dense $O(T^2)$ attention, but only under declared assumptions about their fixed pattern parameters. They trade direct global access for structured communication paths whose adequacy depends on model depth, training, and the task's dependency graph.

FlashAttention remains a distinct idea: it executes dense attention with lower intermediate IO, while sparse masks alter the attention architecture itself. Neither intervention removes every long-context constraint. KV Cache state can continue to grow with sequence length, positional extrapolation can fail beyond training positions, and a large window can still be overwhelmed by irrelevant information. Effective long-context systems therefore combine only the mechanisms justified by their measured compute, memory, positional, and evidence-selection requirements.

# References


[1] R. Child, S. Gray, A. Radford, and I. Sutskever, "Generating Long Sequences with Sparse Transformers," *arXiv preprint arXiv:1904.10509*, 2019, doi: 10.48550/arXiv.1904.10509.

[2] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The Long-Document Transformer," *arXiv preprint arXiv:2004.05150*, 2020, doi: 10.48550/arXiv.2004.05150.

[3] M. Zaheer *et al.*, "Big Bird: Transformers for Longer Sequences," in *Advances in Neural Information Processing Systems 33*, Curran Associates, Inc., 2020. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/hash/c8512d142a2d849725f31a9a7a361ab9-Abstract.html

[4] S. Chen, S. Wong, L. Chen, and Y. Tian, "Extending Context Window of Large Language Models via Positional Interpolation," *arXiv preprint arXiv:2306.15595*, 2023, doi: 10.48550/arXiv.2306.15595.